Apache James

Architecture Analysis & Maintainability Pitch

Future Proof Software Systems
FPSS Exercise – Task 3 • TU Dresden • Summer 2026

6,141	~3.10.0	Apache
Java files	Version	License

Agenda

01 **System overview**
What is James, who uses it, why it exists

02 **High-level architecture**
Modular monolith, layering, event-driven internals

03 **C4 Diagrams**
System, Container, Component and Dynamic Diagrams

04 **Design patterns**
Patterns found — Chain of Responsibility, DI, Observer etc…

05 **Code quality & metrics**
God classes, coupling, test coverage gaps

06 **Testing**
Testing strategies, layers, and module coverage

07 **Improvement proposals**
Prioritised, evidence-backed recommendations

01



System Overview

Java Apache Mail Enterprise Server

Open-source · Apache-licensed · JVM · v3.8.0

System Overview

What is James?

A fully open-source, standards-compliant enterprise mail server. Unifies SMTP, IMAP, POP3, LMTP, JMAP & WebAdmin under one JVM process with pluggable backends.

Who uses it?

ISPs, enterprises needing on-premise mail, open-source mail providers, and developers building automated mail pipelines — newsletters, transactional mail, spam filters.

What problem?

Running enterprise mail typically requires separate tools for relay, IMAP, filtering, and admin. James unifies all with one shared domain model and a rich extension API.

RESPONSIBILITIES

Receive emails	Filter emails
Store emails	Search emails
Deliver emails	Provide administration

Distributed
Cassandra + RabbitMQ + S3 + OpenSearch

PostgreSQL
Postgres + RabbitMQ

JPA / Memory
Embedded HSQLDB — dev & testing only

System Overview

1. server/apps

JAMES starts from these points.

- memory-app
- postgres-app
- distributed-app

Each app says:

“Start James with these modules and these backends.”

- **memory-app**
  - uses in-memory storage
- **postgres-app**
  - uses PostgreSQL
- **distributed-app**
  - uses Cassandra/RabbitMQ/OpenSearch

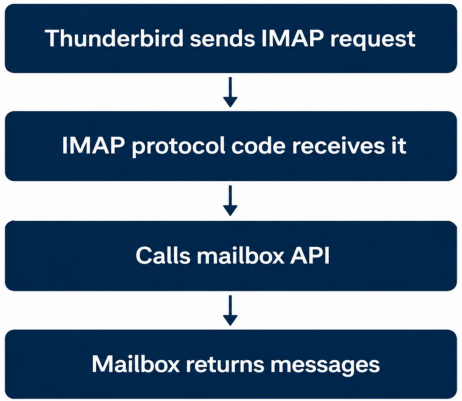
This is the closest thing to the “main application”.

2.protocols / server/protocols

This is the **entry door**. It handles external protocols:

- **SMTP** → receive/send email
- **IMAP** → read/manage mailbox
- **POP3** → download emails
- **JMAP** → JSON mail API
- **LMTP** → local mail delivery
- **WebAdmin** → REST admin API

Example:



System Overview

3. mailet / server/mailet

- This is the **email processing pipeline**.
- A Mailet is like a plugin/filter for emails.
- Mailets decide **what happens to an email after receiving it**.

4. mailbox

- **core email storage logic**

It handles:

- users' mailboxes
- folders
- messages
- quotas
- search
- delete/move/copy messages

5. backends-common

- **This contains shared backend/database code.**
- It is not the business logic. It is infrastructure support.

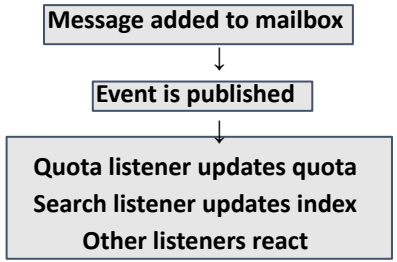
It helps James talk to:

- **PostgreSQL**
- **Cassandra**
- **RabbitMQ**
- **Redis**
- **OpenSearch**
- **Pulsar**

6. event-bus / event-sourcing

- **This is for internal events.**

Example:



System Overview

7. server/task

- This handles background/admin tasks.
- WebAdmin can trigger these tasks.

Examples: reindex mailbox, delete old mails, recompute quota, migrate data, cleanup repository

8. server/webadmin

- This is the REST admin API.

Admins use it to manage:

- users
- domains
- mail queues
- health checks
- tasks
- quotas
- repositories

Admin sends REST request
↓
WebAdmin route/controller
↓
Task/service layer
↓
Mailbox/backend/domain logic

Simple email receive flow:

SMTP request
↓
protocols/server-protocols
↓
mail queue
↓
mailet container
↓
mailbox API
↓
storage backend

Admin flow:

Admin REST request
↓
WebAdmin
↓
Task/API/service
↓
Mailbox or backend

Simple email read flow:

Mail client IMAP/JMAP request
↓
protocol layer
↓
mailbox API
↓
message repository
↓
storage backend

KEY IDEA:

- Protocols accept requests.
- Mailets process mails.
- Mailbox stores mails.
- Backends persist data.
- Event bus connects internal reactions.
- Guice wires everything together.

02



High-Level Architecture

Modular monolith · Event-driven · Plugin-based

Architecture Layers

Protocol servers (Netty NIO)

SMTP · IMAP · POP3 · JMAP · WebAdmin

Mailet container

Chain of Responsibility — Matcher → Mailet pipeline

Mail queue

RabbitMQ / Pulsar / Memory — decouples receipt from delivery

Mailbox layer

StoreMailboxManager · ACL · Quota · Search · Events

Storage backends

Cassandra · PostgreSQL · JPA · S3 · in-memory

Architecture styles

Modular Monolith

All modules in one JVM. 17 Maven modules per concern. Guice DI wires everything.

Event-Driven

EventBus dispatches MailboxEvents. Listeners (IMAP IDLE, quota, search) react without coupling.

Plugin-Based

Mailet/Matcher API loaded from external JARs. 87+ standard mailets. Zero fork needed.

03



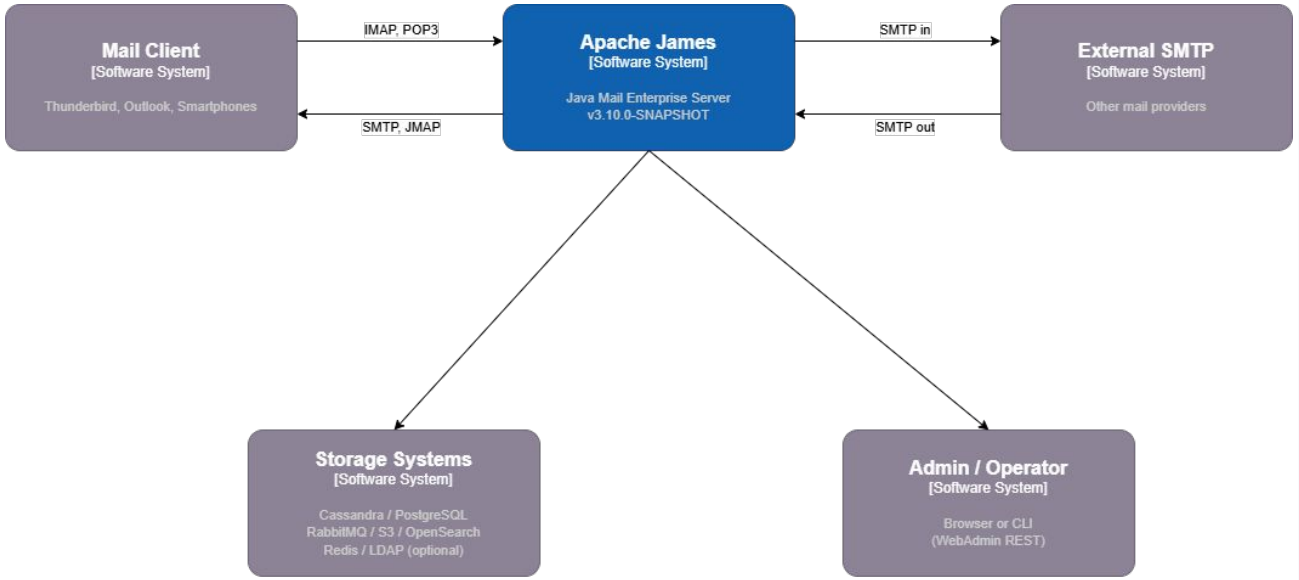
C4 Models

System, Container, Component and Dynamic Diagrams

System Diagram

Flow:

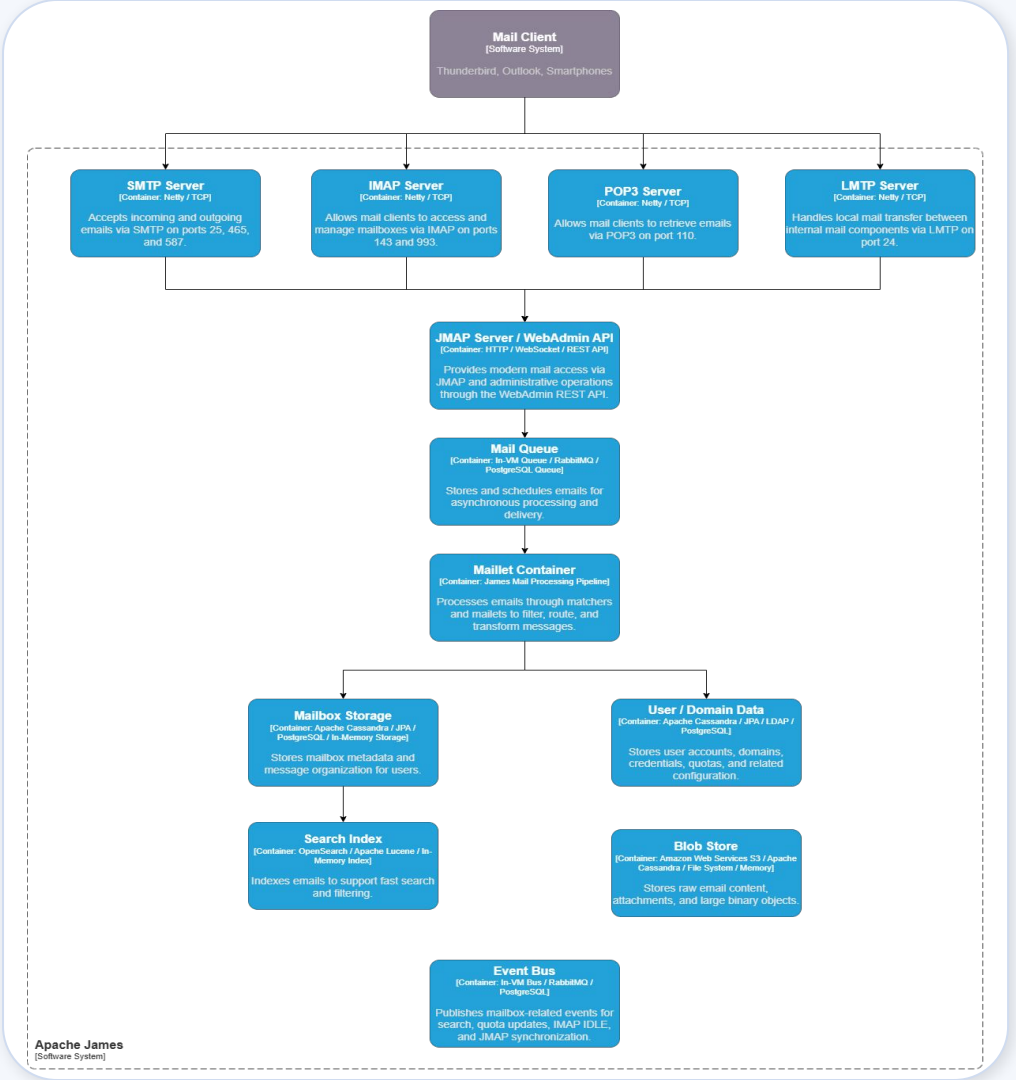
- Apache James is the central mail system.
- Mail Clients send via SMTP/JMAP and read via IMAP/POP3.
- James exchanges mail with External SMTP servers (inbound/outbound).
- James persists data in external Storage Systems
- James is managed by Administrators via WebAdmin REST/CLI.



Container Diagram

Flow:

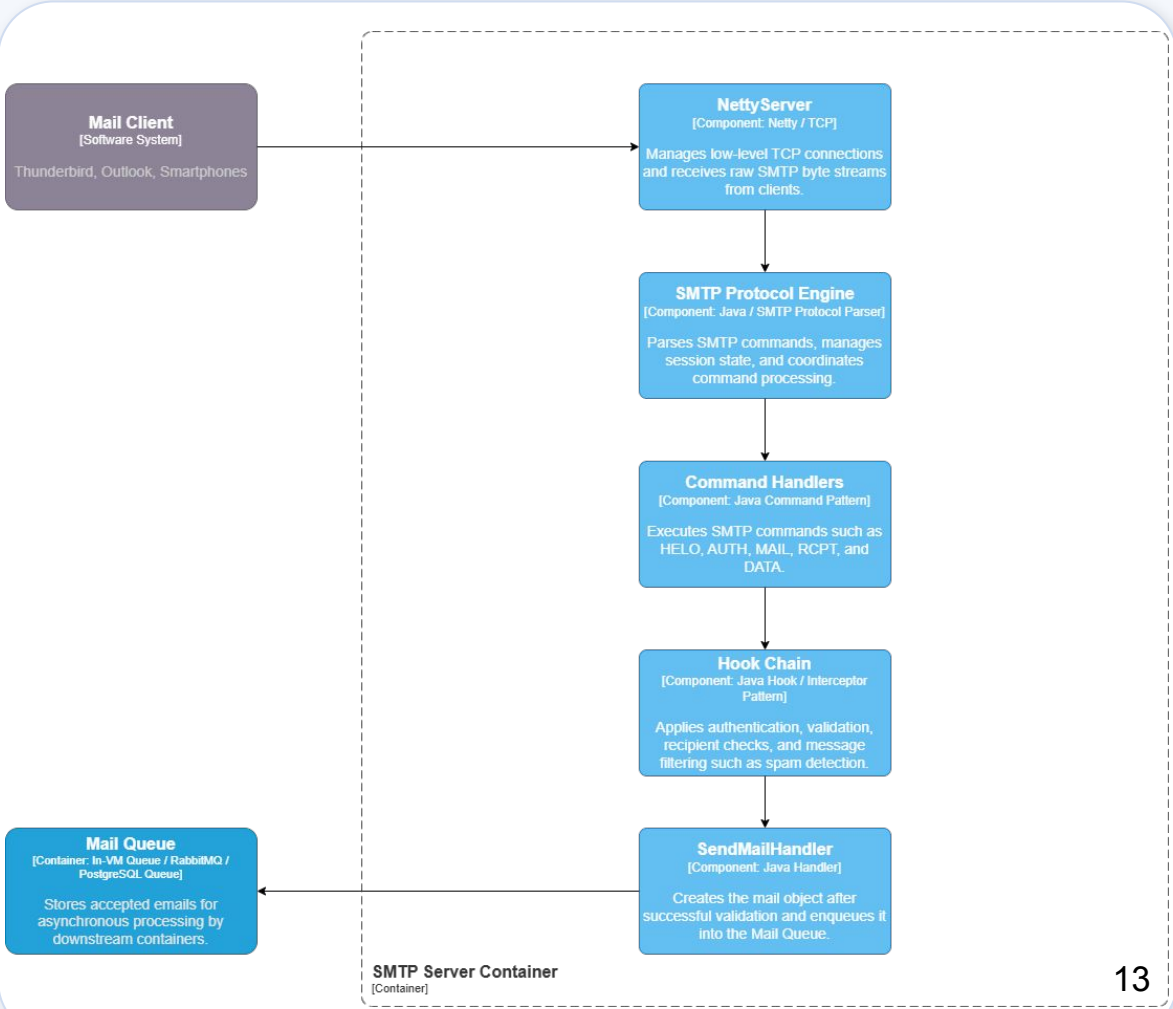
- A mail client or remote server sends a message via SMTP/LMTP.
- The message is placed on the Mail Queue.
- The message is dequeued and processed by the Mailet Container: matchers and mailets handle filtering, routing, and local delivery
- The message is stored across: Mailbox Storage, Search Index, Blob Store
- Mail retrieval via IMAP/JMAP/POP3 reads directly from Mailbox Storage and Search Index, bypassing the queue.
- The Event Bus coordinates mailbox events throughout.



Component Diagram

Flow:

- An external system connects via Netty Server (TCP).
- The connection is passed to the SMTP Protocol Engine.
- Command Handlers process individual SMTP commands.
- The Hook Chain applies: authentication, validation, spam checks
- On success, SendMailHandler dispatches the mail to the Mail Queue for asynchronous downstream processing.



Dynamic Diagram

Flow:

Incoming Email Delivery:

① Mail Client sends via SMTP

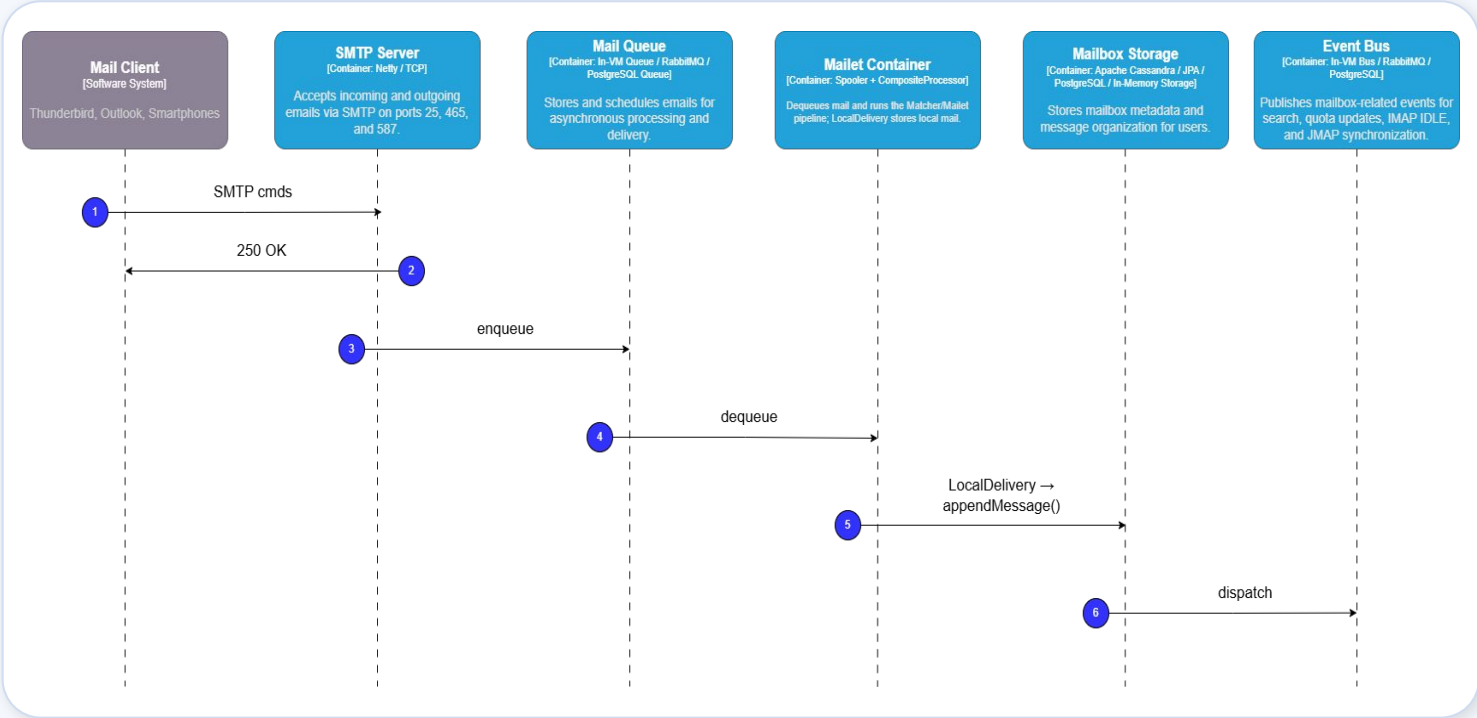
② SMTP Server accepts (250 OK)

③ enqueued in the Mail Queue

④ Mailet Container dequeues

⑤ pipeline processes and LocalDelivery stores it in Mailbox Storage

⑥ events dispatched to the Event Bus for synchronization.



04



Design Patterns

Patterns identified across the codebase

Design Patterns Found

Chain of Responsibility

Mailet container: MatcherSplitter → Mailet chain

Dependency Injection

Guice modules wire protocol, mailbox, queue, and backend implementations

Observer / EventListener

EventBus → OpenSearch indexer, IMAP IDLE notifier, quota listeners

Strategy

MailboxMapper / MessageMapper → swappable storage backends

Repository

MailRepository, UsersRepository, DomainList

Adapter

LuceneMessageSearchIndex / OpenSearchMessageSearchIndex

Factory

MailQueueFactory, SessionFactory

Builder

SearchQuery.Builder, DTO/config builders

Plugin / Extension

Mailet/Matcher API loaded from external JARs

05



Code Quality & Metrics

God classes · Test gaps · Coupling risks identified

JQAssistant Configuration & Limitation

We scanned Apache James with the JQAssistant standalone CLI (`jqassistant scan -f`) — it reads the project directly, without a full Maven build, so it captures the code but not the module wiring.

What was successfully imported

- Maven project structure (POM files)
- Maven modules
- Java packages
- Java classes (Types)
- XML configuration files
- Bytecode metadata

What's missing, and why

- No Maven build means classes are never linked to their modules.
- Packages, classes, and maven modules all exist — but the **module → package/class links are missing**.

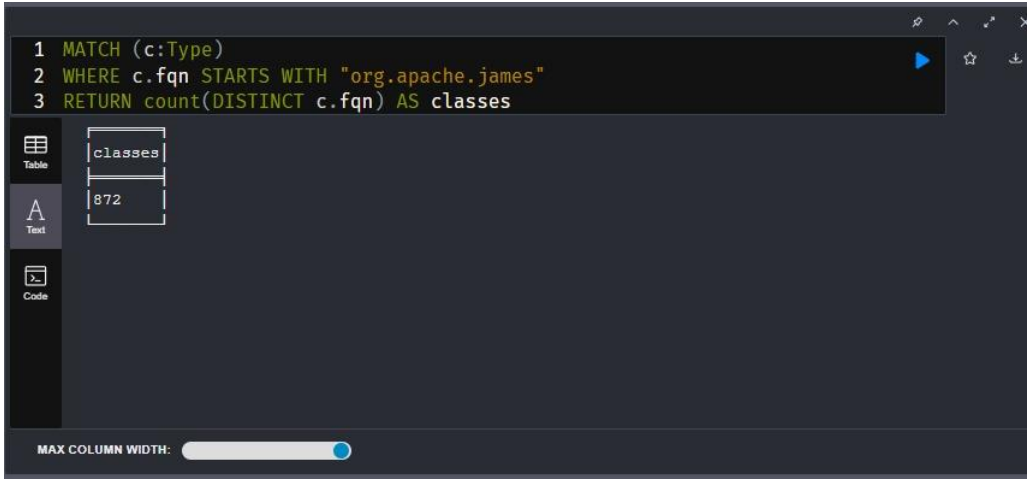
"JQAssistant analysis was performed mainly at package/type/dependency level because Maven module nodes were present but not directly linked to Java types in the scanned graph."

Analysis covers the Java layer (~87% of the codebase). The ~12.5% written in Scala is not parsed by JQAssistant's Java scanner and would require a separate pass.

Still possible:

- Package analysis
- Class dependency analysis
- Fan-in / Fan-out analysis
- Reuse factor analysis
- Inheritance analysis
- God-class identification

1. How many classes?



What it measures

- Distinct classes/interfaces in the `org.apache.james` tree, counted by unique fully-qualified name via JQAssistant.

Why it matters

- Class count is a proxy for codebase size and structural complexity.
- Sets the baseline scope for the coupling and cohesion metrics that follow.

Our finding

- **872 distinct classes** under the `org.apache.james` namespace — a mid-to-large enterprise codebase.

"JQAssistant analysis was performed mainly at package/type/dependency level because Maven module nodes were present but not directly linked to Java types in the scanned graph."

2. Largest package (by class count)

```
1 MATCH (p:Package)-[:CONTAINS]→(c:Type)
2 WHERE p.fqn STARTS WITH "org.apache.james"
3 RETURN p.fqn AS package, count(DISTINCT c.fqn) AS classes
4 ORDER BY classes DESC LIMIT 10
```

Table | Text | Code

package	classes
"org.apache.james.mailbox.model"	139
"org.apache.james.util"	68
"org.apache.james.mime4j.field"	53
"org.apache.james.mailbox"	47
"org.apache.james.mime4j.field.address"	33
"org.apache.james.events"	30
"org.apache.james.mime4j.message"	29
"org.apache.james.task"	28
"org.apache.james.mailbox.exception"	25
"org.apache.james.core.quota"	25

What it measures

- Classes/interfaces directly within each package, which packages carry the most structural weight.

Why it matters

- Large packages often signal low cohesion or a "god package" doing too much.
- Highlights candidates for refactoring or splitting, and where to focus deeper analysis.

Our finding

- **org.apache.james.mailbox.model** is largest at **139 classes** — over double the runner-up (**util, 68**).
- Mailbox packages dominate the top 10, reflecting the core domain focus.

3. Package with most lines of code

```
1 MATCH (p:Package)-[:CONTAINS]→(c:Type)-[:DECLARES]→(m:Method)
2 WHERE p.fqn STARTS WITH "org.apache.james"
3 RETURN p.fqn AS package, sum(m.effectiveLineCount) AS totalLoc
4 ORDER BY totalLoc DESC LIMIT 10
```

Table | Text | Code

package	totalLoc
"org.apache.james.util"	2722
"org.apache.james.mailbox.model"	2185
"org.apache.james.mime4j.field.address"	1938
"org.apache.james.mime4j.message"	1157
"org.apache.james.mime4j.field.datetime.parser"	1153
"org.apache.james.core"	1138
"org.apache.james.mime4j.field"	1056
"org.apache.james.task"	1029
"org.apache.james.mime4j.stream"	971
"org.apache.james.mime4j.field.contenttype.parser"	908

What it measures

- Effective lines of code (method bodies) summed across all classes in each package.

Why it matters

- Complements class count, a package can have few classes but heavy logic.
- High LOC flags packages that are harder to test and maintain, and spots complexity hiding in few classes.

Our finding

- **org.apache.james.util** has the most code at **2,722 LOC**, despite ranking 2nd in class count, denser, more complex classes.
- **mailbox.model** follows at **2,185 LOC**, consistent with its high class count.

4. Most reused components: fan-in

Most reused = many other classes depend on it.

```
MATCH (user:Type)-[:DEPENDS_ON]→(target:Type)
WHERE user.fqn STARTS WITH "org.apache.james"
  AND target.fqn STARTS WITH "org.apache.james"
  AND user <> target
RETURN target.fqn AS reusedComponent,
       count(DISTINCT user) AS reusedBy
ORDER BY reusedBy DESC
LIMIT 20;
```

Table | Text | Code

reusedComponent	reusedBy
"org.apache.james.mime4j.stream.Field"	78
"org.apache.james.mime4j.codec.DecodeMonitor"	76
"org.apache.james.mime4j.dom.FieldParser"	60
"org.apache.james.mime4j.dom.field.ParsedField"	52
"org.apache.james.core.Username"	46
"org.apache.james.mailbox.exception.MailboxException"	46
"org.apache.james.mailbox.model.MailboxId"	39
"org.apache.james.mailbox.model.MailboxPath"	32
"org.apache.james.mime4j.MimeException"	28
"org.apache.james.mailbox.model.MessageId"	26

What it measures

- Shows **which classes are reused by the largest number of other classes**.
- A high **Fan-in** means the component is **central and widely shared** across the system.

Why it matters

- High reuse usually indicates an important framework or domain component.
- Changes to these classes can affect many other components.

Our finding

- Classes such as **Field**, **DecodeMonitor**, and **Username** are among the most reused components in Apache James.

5. Coupling analysis: fan-out

Most coupled = depends on many other classes/packages.

```
MATCH (source:Type)-[:DEPENDS_ON]→(target:Type)
WHERE source.fqn STARTS WITH "org.apache.james"
  AND target.fqn STARTS WITH "org.apache.james"
  AND source <> target
RETURN source.fqn AS component,
       count(DISTINCT target) AS dependsOn
ORDER BY dependsOn DESC
LIMIT 20;
```

Table | Text | Code

component	dependsOn
"org.apache.james.mailbox.model.SearchQuery"	40
"org.apache.james.mime4j.dom.Message$Builder"	40
"org.apache.james.mime4j.message.DefaultMessageBuilder"	32
"org.apache.james.mime4j.stream.MimeEntity"	29
"org.apache.james.mime4j.field.Fields"	29
"org.apache.james.server.task.json.dto.TestTaskDTOModules"	27
"org.apache.james.mailbox.MessageManager"	25
"org.apache.james.mailbox.MailboxManager"	24
"org.apache.james.mime4j.field.address.AddressListParser"	24
"org.apache.james.mime4j.field.address.Builder"	23

What it measures

- Shows **how many other classes a component depends on**.
- A high **Fan-out** means the class interacts with many other parts of the system.

Why it matters

- High coupling increases maintenance complexity.
- Changes in dependencies are more likely to impact these classes.

Our finding

- **SearchQuery**, **MessageBuilder**, and **MailboxManager** have some of the highest dependency counts.

6. Reuse factor

```
MATCH (p:Package)
WHERE p.fqn STARTS WITH "org.apache.james"
OPTIONAL MATCH (inPkg:Package)-[:CONTAINS]→(:Type)-[:DEPENDS_ON]→
(:Type)←[:CONTAINS]-(p)
WHERE inPkg.fqn STARTS WITH "org.apache.james" AND inPkg <> p
WITH p, count(DISTINCT inPkg) AS fanIn
OPTIONAL MATCH (p)-[:CONTAINS]→(:Type)-[:DEPENDS_ON]→(:Type)←
[:CONTAINS]-(outPkg:Package)
WHERE outPkg.fqn STARTS WITH "org.apache.james" AND outPkg <> p
WITH p, fanIn, count(DISTINCT outPkg) AS fanOut
RETURN p.fqn AS package,
       fanIn,
```

Table | Text | Code

package	fanIn	fanOut	reuseFactor
"org.apache.james.mime4j"	13	0	13
"org.apache.james.mime4j.dom.field"	12	3	4.0
"org.apache.james.core"	11	0	11
"org.apache.james.mime4j.util"	11	2	5.5
"org.apache.james.mailbox.model"	9	12	0.75
"org.apache.james.mime4j.stream"	9	4	2.25
"org.apache.james.mime4j.codec"	8	3	2.67
"org.apache.james.mailbox"	8	7	1.14
"org.apache.james.mime4j.dom"	7	12	0.58
"org.apache.james.mailbox.exception"	7	4	1.75

MAX COLUMN WIDTH:

Reuse factor = Fan-in / Fan-out

High value = many packages reuse it, but it depends on few packages.

What it measures

- Evaluates how effectively a component is reused compared to how many dependencies it has.

Interpretation

- **High value**
  - Highly reusable
  - Low dependency
  - Better modular design
- **Low value**
  - Depends on many components
  - Less reusable
  - Higher maintenance effort

Our finding

- Packages such as **mime4j** and **core** exhibit high reuse factors, indicating good modularity.

7. Inheritance Depth

```
MATCH path = (child:Type)-[:EXTENDS*1..10]→(parent:Type)
WHERE child.fqn STARTS WITH "org.apache.james"
RETURN child.fqn AS type,
       max(length(path)) AS inheritanceDepth
ORDER BY inheritanceDepth DESC
LIMIT 20;
```

Table | Text | Code

type	inheritanceDepth
"org.apache.james.mailbox.events.MailboxEvents$Expunged"	4
"org.apache.james.mailbox.events.MailboxEvents$Added"	4
"org.apache.james.mime4j.field.address.ASTlocal_part"	3
"org.apache.james.mailbox.exception.UnsupportedRightException"	3
"org.apache.james.mime4j.field.address.ASTaddress_list"	3
"org.apache.james.mime4j.field.address.ASTgroup_body"	3
"org.apache.james.mailbox.exception.InsufficientRightsException"	3
"org.apache.james.mailbox.events.MailboxEvents$FlagsUpdated"	3
"org.apache.james.javax.MimeMultipartReport"	3
"org.apache.james.mailbox.events.MailboxEvents$MetaDataHoldingEvent"	3

What it measures

- Calculates the **maximum inheritance chain** from a class to its root superclass.

Why it matters

- Deep inheritance increases complexity.
- Shallow inheritance is generally easier to understand and maintain.

Our finding

- The deepest inheritance observed is **4 levels**, suggesting Apache James uses inheritance conservatively.

8. God-Class Candidates by method count

```
1 MATCH (c:Type)-[:DECLARES]→(m:Method)
2 WHERE c.fqn STARTS WITH "org.apache.james"
3 RETURN c.fqn AS class,
4        count(m) AS methods
5 ORDER BY methods DESC
6 LIMIT 20;
```

Table | Text | Code

class	methods
"org.apache.james.mdn.MDNReportParser"	330
"org.apache.james.mime4j.dom.Message$Builder"	123
"org.apache.james.task.TaskExecutionDetails"	109
"org.apache.james.mime4j.message.MessageBuilder"	85
"org.apache.james.util.ReactorUtils"	84
"org.apache.james.core.builder.MimeMessageBuilder"	77
"org.apache.james.core.MailAddress"	76
"org.apache.james.task.TaskManagerContract"	72
"org.apache.james.core.UsernameTest"	63
"org.apache.james.mailbox.model.SearchQuery"	60

What it measures

- Identifies classes containing a very large number of methods.

Why it matters

- Classes with many methods may violate the **Single Responsibility Principle (SRP)**.
- They are harder to understand, test, and maintain.

Our finding

- **MDNReportParser** (330 methods),
- **MessageBuilder**,
- **TaskExecutionDetails**

are the largest classes by method count and are candidates for further architectural review.

9. Interface Ratio

```
1
2 MATCH (c:Java:Type:Class) WITH count(c) AS classes
3 MATCH (i:Java:Type:Interface) WITH classes, count(i) AS interfaces
4 RETURN classes, interfaces,
5        round(100.0 * interfaces / classes) AS interfaceRatio;
```

Table | Text | Code

	classes	interfaces	interfaceRatio
1	9859	1947	20.0

What it measures

- Ratio of interfaces to classes across the scanned graph, how much the code relies on abstraction.

Why it matters

- A healthy interface ratio signals good use of abstraction and pluggable, swappable implementations.
- Too few interfaces can mean tight coupling; the ratio hints at design flexibility.

Our finding

- **9,859 classes, 1,947 interfaces; a 20% ratio** (1 interface per 5 classes).
- A solid level of abstraction, consistent with James's pluggable-backend design.

Counted across the full scanned graph, including bundled libraries.

Code Quality

Existing aspects vs. limitations — evidence-based

Existing Aspects (strengths):

- **Clear layered architecture** — protocols → mailet → mailbox → backends, each with a distinct responsibility
- **Conservative inheritance** — deepest chain only **4 levels**, easy to follow
- **Strong modularity in shared packages** — `core` shows high reuse factor (highly reused, low dependency)
- **Consistent contract pattern** across storage backends (Cassandra/Postgres/Memory) reduces duplicated logic

Limitations:

- **God-class candidates** — `MDNReportParser` has **330 methods**, far beyond a maintainable size for one class
- **High coupling in central classes** — `SearchQuery` and `MessageBuilder` depend on **~40 other types** each; `MailboxManager` ~24
- **Heavily reused core classes** — `Username` reused by 46+ classes; changes ripple across dozens of dependents (high-risk surface)
- **No automated architecture enforcement** (e.g. ArchUnit) — layering is convention, not tested
- **Multiple storage backends** (Cassandra/Postgres/JPA/Memory/S3) increase configuration and testing burden

Code Quality
Scalability / Maintainability

- **Built for scale**: modular monolith, pluggable backends; scales dev → enterprise without a rewrite
- **Healthy structure (measured)**: inheritance only **4 levels** deep; `core` shows high reuse factor → good modularity
- **But maintainability is uneven (measured)**: god-classes (`MDNReportParser` 330 methods, `Message$Builder` 123) and coupled classes (`SearchQuery`/`MessageBuilder` ~40 deps) concentrate risk
- **Contract tests** triple validation cost, every feature checked across Cassandra/Postgres/Memory
- **To improve**: split god-classes, cut fan-out, add ArchUnit to enforce layering

06



Testing

Testing Analysis

Apache James — evidence from the codebase

What is tested?

The test suite covers the full mail pipeline, not just isolated logic:

- Protocol correctness (SMTP, IMAP, JMAP, WebAdmin)
- Mailbox storage behavior (folders, messages, quotas, search)
- Mailet processing pipeline (matchers, filters, transformations)
- Cross-backend consistency (Cassandra / PostgreSQL / Memory)

Focus is heavily on the core "receive → process → store" path. `mailbox/store` (492 test files) and `protocols/imap` (99 files) are the most heavily tested modules.

How is it tested?

- **Unit Testing:** Fast, isolated class-level validation without DB or network dependencies.
- **Contract Testing:** Reusable test suites run against multiple backends (Cassandra/Postgres/Memory) to ensure identical behavior.
- **Integration Testing:** `server/mailet/integration-testing` alone has 93 test files; JMAP is tested separately per backend (56 memory / 48 distributed / 38 postgres)
- **Infrastructure Testing:** 41 files use Testcontainers to spin up real Cassandra/RabbitMQ/S3 in Docker, not mocks

Server Tests and Protocol Tests

Two distinct testing layers, with real setup cost

Protocol Tests — `protocols/` + `mpt/`

- Live in `protocols/` — test each protocol on its own: IMAP (99 files), POP3 (20), SMTP (18), LMTP (7)
- James also has its own protocol test framework, **MPT (Mail Protocol Tester)**, with 221 test files. It scripts and replays real IMAP/SMTP/ManageSieve conversations. Most projects don't build this themselves.
- These run without a full server boot — faster, more isolated, easier to pinpoint a failing parser or command handler
- Total: protocols/ + mpt = 320 test files

Server Tests — `server/protocols/` + `server/apps/`

- Live in `server/protocols/` and `server/apps/` — test the whole assembled application, not just one parser
- WebAdmin (REST API): 147 + 41 test files
- JMAP integration tests: 142 files, split across memory, distributed, and postgres backends
- Each deployable app (`memory-app`, `postgres-app`, `distributed-app`, etc.) has its own boot/integration tests
- `server/` totals 1,332 test files — over half the project

Test Coverage by Module

Module	Test files	Focus
protocols/imap	99	IMAP protocol correctness
server/mailet/integration-testing	93	End-to-end mail flow
mailbox/cassandra	84	Cassandra storage behavior
mailbox/store	83	Core mailbox storage logic
mailet/standard	82	Standard mailet implementations
mailbox/api	77	Mailbox API contracts
server/protocols/webadmin	188	REST admin API
JMAP rfc-8621 (memory/distributed/postgres)	56 / 48 / 38	JMAP per backend
mpt (protocol conformance)	221	Scripted protocol conversations
Testcontainers-based files (codebase-wide)	41	Real Docker infra

Risk Assessment

Apache James is architecturally strong and highly extensible, but its flexibility creates maintainability risks. The main concern is not a bad architecture, but the **complexity of managing many modules, protocols, backends, and extension points over time**.

Risk Area	Risk Level	Explanation
Build complexity	High	Large Maven multi-module project; difficult for new developers to build and analyze quickly.
Architecture complexity	High	Many protocols, storage backends, Guice modules, and deployment variants increase onboarding effort.
Large classes / God-class candidates	Medium–High	Classes like MDNReportParser, MessageBuilder, SearchQuery, and mailbox managers have many methods/responsibilities.
High coupling	Medium–High	Some central classes/packages depend on many internal types, making changes harder and riskier.
Storage backend complexity	Medium–High	Multiple backends such as Memory, PostgreSQL, Cassandra, JPA, OpenSearch, RabbitMQ, and S3 increase configuration and testing burden.

Strengths and Weaknesses

Strength	Evidence
Clean layer separation	Protocol → Queue → Mailet → Storage are fully decoupled; SMTP server never touches mailbox directly
Pluggable backends	MailboxManager, BlobStore, MessageSearchIndex are interfaces; Cassandra/JPA/PostgreSQL/memory can be swapped
Powerful extension point	Mailet API allows third-party JARs to inject mail logic without core changes
Strong test coverage	2,559 test files; 88 contract tests; 221 MPT protocol integration tests
Event-driven side effects	EventBus decouples quota, indexing, and IMAP push
Non-blocking I/O	Netty + Reactor (Flux/Mono) handle high concurrency
Multiple deployment flavors	Runs from in-memory testing to Cassandra + RabbitMQ + S3

Strengths and Weaknesses

Weakness	Evidence
God classes	StoreMailboxManager 1212 LOC, MessageProcessor 1659 LOC
High coupling	Storage hub imports 83 classes
Legacy monolith	FetchMail code has low cohesion
XML config not type-safe	Errors appear only at startup/runtime
Opaque Guice wiring	Dependency tracing is difficult
Oversized IMAP package	302 Java files, hard to navigate
Weak module boundaries	Internal imports bypass APIs
Documentation gaps	Overview docs incomplete

07



Improvement Proposals

Prioritised & evidence-backed recommendations

Future Improvements

1) Code Quality & Maintainability

- Enforce consistent static analysis (Checkstyle/Spotless/SonarQube) across all modules to catch style drift in such a large multi-module codebase
- Reduce module coupling — break down god-modules and tightly-coupled mailet/matcher implementations into smaller, independently testable units
- Adopt architecture fitness functions (e.g. ArchUnit) to automatically catch architecture violations in CI before they accumulate

2) Scalability & Architecture

- Evaluate further decoupling between storage backends (Cassandra/Postgres/JPA) to simplify supporting/deprecating backends over time
- Introduce clearer module boundaries/domain-driven design boundaries to make onboarding new contributors faster
- Add observability (metrics/tracing) consistently across all mailets/matchers, not just core modules

Future Improvements

3) Testing & Integration

- Increase integration test coverage for less-tested modules (e.g. newer Postgres-backed components, JMAP RFC-8621 integration)
- Introduce mutation testing (PIT) to validate the real strength of existing unit tests, not just coverage percentage
- Add contract tests between modules/APIs to prevent silent breakage when one module changes

4) Dependency & Build Management

- Reduce dependency sprawl — audit and consolidate overlapping libraries (e.g. multiple logging/serialization libs) to cut build fragility and security surface
- Automate dependency vulnerability scanning (OWASP Dependency-Check / Snyk) given the project's age and dependency count
- Modularize the Maven build further to reduce full-rebuild times, improving developer velocity

Thank You

Apache James — Architecture Analysis

FPSS Exercise Task-3 · TU Dresden · Summer 2026